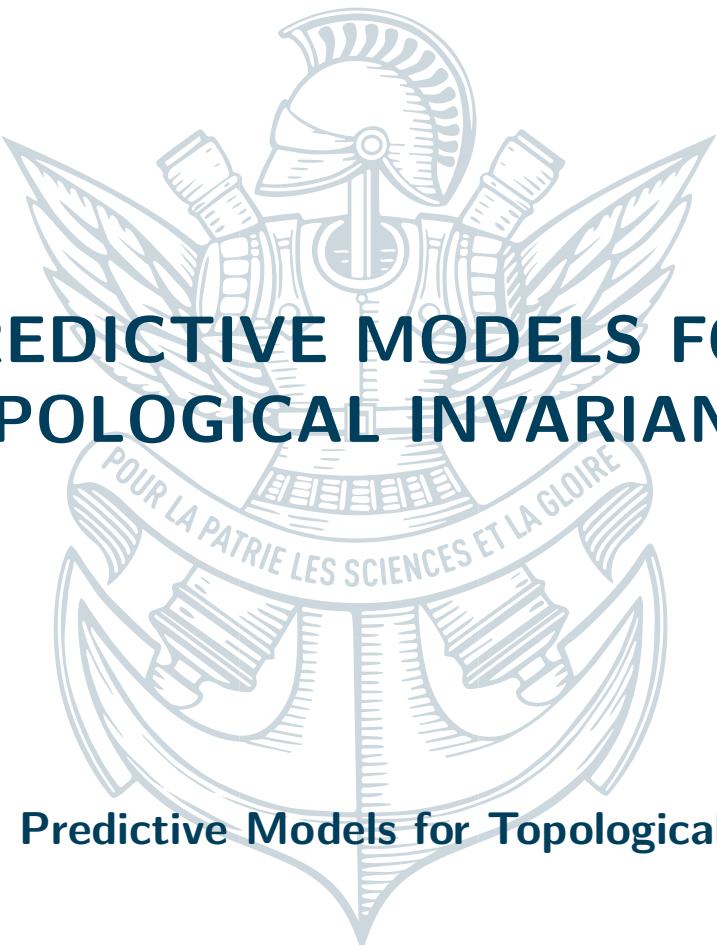




PREDICTIVE MODELS FOR TOPOLOGICAL INVARIANTS

Project 8 : Predictive Models for Topological Invariants

July 9, 2026

Ten Nguyen Hanaoka



Abstract

We study predictive models for topological invariants in Topological Data Analysis, with a focus on point-cloud learning. The central object is the persistence diagram and its vectorizations, which provide stable summaries of geometric structure but remain expensive to compute exactly. Motivated by RipsNet, which learns persistence-based descriptors directly from unordered point sets, we investigate a broader family of symmetry-aware models. The project developed in three stages: first `DistanceMatrixRaggedModel`, then `PointNet3D`, and finally a family of Tensor Field Network (TFN) architectures generalized to dimension n . This final step was motivated by the desire to obtain robustness to both point permutations and rigid isometries, which the earlier models did not fully provide. The main contribution of the paper is this TFN family for geometric learning on 3D and higher-dimensional point clouds, together with the two earlier baselines. We also describe the PyTorch implementation and experimental workflow used to compare these models on synthetic and UCR benchmarks under clean, noisy, and symmetry-perturbed settings.

CONTENTS

1	Introduction	3
2	Background	3
2.1	Persistence Diagrams	3
2.2	Bottleneck Distance and Stability	4
2.3	Vectorizations of Persistence Diagrams	4
2.4	Permutation-Invariant Set Functions	5
2.5	Group Actions and Equivariance	5
2.6	Representations and Tensor Products	5
2.7	Clebsch-Gordan Coefficients	6
2.8	Local Geometric Encodings	6
3	Related Work	6
4	Models	7
4.1	<code>DistanceMatrixRaggedModel</code>	7
4.2	<code>PointNet3D</code>	8
4.3	Tensor Field Networks	8
5	Experimental Setup	9
5.1	Evaluation Protocol	10
5.2	Synthetic Point-Cloud Benchmarks	10
5.3	UCR Benchmarks	10
5.4	Training Scripts	10
5.5	Experiment Directory	11
5.6	Notebook Workflow	11
6	Results	11
7	Conclusion	12

1

INTRODUCTION

Topological Data Analysis studies data through topological invariants such as persistence diagrams. These descriptors are informative and stable, but their exact computation requires building filtered simplicial complexes and computing persistent homology, which can become expensive for large or dense point clouds.

Learning-based approaches seek to approximate these descriptors directly from the input geometry. RipsNet [14] is a representative example: it maps unordered point clouds to vectorized persistence descriptors using a permutation-invariant neural architecture. The project behind this paper originally began from a PyTorch reimplementation of that line of work and then expanded toward symmetry-aware geometric models that could better exploit the structure of point clouds.

The project developed in three stages. We first introduced `DistanceMatrixRaggedModel`, a persistence-oriented model based on pairwise distances. We then added `PointNet3D` as a permutation-invariant baseline. Finally, we generalized the TFN family to arbitrary ambient dimension n in order to target robustness to both permutations and isometries more directly than before.

The resulting study therefore compares three families of models in their historical order of development. First comes `DistanceMatrixRaggedModel`. Second comes `PointNet3D`. Third comes the TFN family, which is the most symmetry-aware part of the project and the one that motivated the higher-dimensional generalization.

The empirical evaluation is carried out on two complementary testbeds. Synthetic point-cloud benchmarks probe whether the models recover topological information under noise and rigid transformations, while a suite of UCR datasets provides a broader benchmark for robustness and sample efficiency. Together, these experiments make it possible to study how different inductive biases behave when the input geometry, ordering, and noise level vary.

The remainder of the paper is organized as follows. Section 2 collects the mathematical preliminaries, Section 3 reviews related work, Section 4 presents the model families, Section 5 describes the experimental setup, and Section 6 discusses the results.

2

BACKGROUND

We collect the mathematical preliminaries used throughout the paper. The first part recalls persistent homology and the standard vectorizations of persistence diagrams. The second part recalls permutation-invariant set functions, group actions, and the representation-theoretic language used to describe symmetry-aware models on point clouds.

2.1 PERSISTENCE DIAGRAMS

Filtered complexes. Let $(\mathcal{K}_t)_{t \in \mathbb{R}}$ be a filtration of simplicial complexes, meaning that $\mathcal{K}_s \subseteq \mathcal{K}_t$ whenever $s \leq t$. Applying homology in degree k gives a persistence module $(H_k(\mathcal{K}_t))_{t \in \mathbb{R}}$. The

associated persistence diagram is the multiset

$$D_k = \{(b_i, d_i)\}_{i=1}^m \subset \{(b, d) \in \mathbb{R}^2 : b < d\},$$

where each point records the birth and death times of a homology class.

Vietoris-Rips filtration. For a point cloud $\mathcal{P} = \{p_1, \dots, p_n\} \subset \mathbb{R}^d$, the Vietoris-Rips complex at scale t is

$$\text{VR}_t(\mathcal{P}) = \{\sigma \subseteq \mathcal{P} : \|p_i - p_j\|_2 \leq t \text{ for all } p_i, p_j \in \sigma\}.$$

Equivalently, if $d(x, \mathcal{P}) = \min_{p \in \mathcal{P}} \|x - p\|_2$, then the sublevel sets of $d(\cdot, \mathcal{P})$ generate a filtration by unions of balls. Because $\text{VR}_t(\mathcal{P})$ depends only on pairwise distances, it is invariant under rigid motions of $\mathbb{R}^d$.

2.2 BOTTLENECK DISTANCE AND STABILITY

The bottleneck distance between two persistence diagrams D_1 and D_2 is

$$W_\infty(D_1, D_2) = \inf_{\gamma} \sup_{x \in D_1} \|x - \gamma(x)\|_\infty,$$

where γ ranges over bijections between $D_1 \cup \Delta$ and $D_2 \cup \Delta$, with $\Delta = \{(t, t) : t \in \mathbb{R}\}$ the diagonal. Informally, one is allowed to match points to each other or to the diagonal.

The fundamental stability statement is that small perturbations of the input induce small perturbations of the diagram. For tame functions f, g on the same domain, one has the estimate

$$W_\infty(D_k(f), D_k(g)) \leq \|f - g\|_\infty.$$

This stability property is one of the main reasons persistence-based descriptors are useful in learning.

2.3 VECTORIZATIONS OF PERSISTENCE DIAGRAMS

Persistence diagrams live in a nonlinear space, so they are often mapped to finite-dimensional vectors. Two standard constructions are persistence images and persistence landscapes.

Persistence images. Let $D = \{(b_i, d_i)\}_{i=1}^m$. After the change of variables $(b, d) \mapsto (b, d - b)$, a weighted persistence surface can be written as

$$\rho_D(x, y) = \sum_{i=1}^m w(b_i, d_i) K_\sigma((x, y) - (b_i, d_i - b_i)),$$

where w is a weight function and K_σ is a Gaussian kernel. The persistence image is obtained by integrating ρ_D over a fixed grid of pixels.

Persistence landscapes. For each point $(b, d) \in D$, define the tent function

$$\Lambda_{(b,d)}(t) = \max\{0, \min(t - b, d - t)\}.$$

The persistence landscape is the sequence $\lambda_k(t)$, where $\lambda_k(t)$ is the k -th largest value among the set $\{\Lambda_{(b_i, d_i)}(t)\}_{i=1}^m$ at each $t \in \mathbb{R}$. Sampling the functions λ_k on a grid gives a finite vector representation.

2.4 PERMUTATION-INVARIANT SET FUNCTIONS

A finite point cloud can be viewed as an unordered multiset

$$X = \{x_1, \dots, x_n\} \subset \mathbb{R}^d.$$

A function F on such inputs is permutation-invariant if $F(X) = F(\pi X)$ for every permutation $\pi \in S_n$. The Deep Sets theorem shows that many continuous invariant functions admit the form

$$F(X) = \rho\left(\sum_{i=1}^n \phi(x_i)\right),$$

for suitable maps ϕ and ρ [1]. The sum can be replaced by another symmetric reduction, such as max pooling, when one wants a different aggregation rule; this is the structural idea behind PointNet [2]. In practice, ragged batches are implemented by masking padded entries before the symmetric pooling step. This same viewpoint also explains why a row-wise encoder applied to a pairwise distance matrix can still produce a permutation-invariant prediction after symmetric aggregation.

2.5 GROUP ACTIONS AND EQUIVARIANCE

Let G be a group acting on a set X , and let $\rho : G \rightarrow GL(V)$ be a linear representation on a vector space V . A map $F : X \rightarrow V$ is *equivariant* if

$$F(g \cdot x) = \rho(g)F(x) \quad \text{for all } g \in G, x \in X.$$

It is *invariant* if $\rho(g) = I_V$ for every $g \in G$.

For point clouds, a second natural symmetry is permutation of the input ordering. If $\pi \in S_n$ acts by reindexing the points of $P = (p_1, \dots, p_n)$, then a permutation-invariant model satisfies

$$F(p_1, \dots, p_n) = F(p_{\pi(1)}, \dots, p_{\pi(n)}).$$

2.6 REPRESENTATIONS AND TENSOR PRODUCTS

Many symmetry-aware models organize their hidden states according to representation type. A finite-dimensional representation of a group G is a vector space V together with a homomorphism $\rho : G \rightarrow GL(V)$. When V is reducible, it decomposes as a direct sum of irreducible representations:

$$V \cong \bigoplus_{\lambda} m_{\lambda} V_{\lambda}.$$

For $SO(3)$, the irreducible representations are indexed by $\ell \in \mathbb{N}$ and have dimension $2\ell + 1$. Their tensor products decompose according to

$$V_{\ell_1} \otimes V_{\ell_2} \cong \bigoplus_{\ell=|\ell_1-\ell_2|}^{\ell_1+\ell_2} V_{\ell}.$$

This decomposition is the algebraic reason tensor features can be coupled while preserving symmetry. For higher-dimensional ambient spaces, the same idea applies after replacing $SO(3)$ by the symmetry group appropriate to the problem, typically $SO(n)$ or $O(n)$. The irreducible blocks and their bases then depend on n , but the guiding principle remains the same: decompose tensor products into symmetry-adapted components and keep track of the corresponding intertwiners.

2.7 CLEBSCH-GORDAN COEFFICIENTS

Clebsch-Gordan coefficients are the change-of-basis constants associated with the above decomposition. If $\{e_{m_1}^{\ell_1}\}$, $\{e_{m_2}^{\ell_2}\}$, and $\{e_m^\ell\}$ are orthonormal bases of the corresponding irreducible spaces, then

$$e_{m_1}^{\ell_1} \otimes e_{m_2}^{\ell_2} = \sum_{\ell, m} C_{m_1, m_2, m}^{\ell_1, \ell_2, \ell} e_m^\ell.$$

The coefficients $C_{m_1, m_2, m}^{\ell_1, \ell_2, \ell}$ encode how two representations can be coupled into a third one. More generally, if

$$V_\alpha \otimes V_\beta \cong \bigoplus_{\gamma} N_{\alpha\beta}^\gamma V_\gamma,$$

then the Clebsch-Gordan coefficients are the entries of the change-of-basis map from the product basis to an adapted irreducible basis. For compact groups, one numerical route is to block-diagonalize the tensor-product representation and extract the intertwining basis [10]. For $SU(N)$ and $SL(N, \mathbb{C})$, the Gelfand-Tsetlin pattern calculus gives an explicit algorithm [11]. In higher-rank Cartesian tensor settings, recent decomposition methods provide scalable orthogonal bases for equivariant spaces and are especially relevant when one wants to generalize the construction from 3D to dimension n [12].

2.8 LOCAL GEOMETRIC ENCODINGS

Let $P = \{x_1, \dots, x_n\} \subset \mathbb{R}^d$ be a point cloud. Its k -nearest-neighbor graph is

$$G_k(P) = (V, E), \quad V = \{1, \dots, n\}, \quad E = \{(i, j) : j \in \mathcal{N}_k(i)\}.$$

For each i , let $\mathcal{N}_k(i)$ denote the set of its k nearest neighbors. For $j \in \mathcal{N}_k(i)$, define the relative vector

$$r_{ij} = x_j - x_i, \quad d_{ij} = \|r_{ij}\|_2, \quad \hat{r}_{ij} = \frac{r_{ij}}{\|r_{ij}\|_2}.$$

A radial basis expansion of the distance can be written as

$$\phi_m(d) = \exp\left(-\frac{(d - \mu_m)^2}{\sigma^2}\right), \quad m = 1, \dots, M,$$

for centers μ_m and width $\sigma > 0$. These radial and directional quantities form the local geometric data used by the models studied later in the paper. They are the basic ingredients of the graph-based message-passing layers in the TFN family, where reindexing the points only relabels the graph nodes.

3 RELATED WORK

Learning with topological features has developed along several complementary directions. Early work focused on turning persistence diagrams into vector spaces through constructions such as persistence images and persistence landscapes, making them compatible with standard neural architectures and downstream classifiers [5, 6]. The stability theorem for persistence diagrams provides the mathematical

justification for these constructions [7], and later work studied perturbation robustness and optimized persistence-based losses and layers [15, 16, 17, 18].

Another line of research learns topological or geometric signatures directly from point clouds. RipsNet [14] is a representative example: it predicts vectorized persistence descriptors from unordered point sets using a Deep Sets-style architecture. In a similar spirit, the Deep Sets theorem [1] and PointNet [2] show how permutation invariance can be built into the architecture through shared pointwise encoders and symmetric pooling. Later point-cloud baselines such as PointNet++ [3] and DGCNN [4] improve on this idea by adding local neighborhoods and graph-based feature extraction.

Symmetry-aware geometric learning extends these ideas from permutation invariance to rigid-motion equivariance. Tensor Field Networks [8] are a central reference point in this literature because they couple point coordinates and tensor features through equivariant tensor products. More recent work on any-dimensional equivariant neural networks [9] provides a principled route for extending such constructions beyond the 3D setting, which is exactly the direction taken by the TFN variants developed in this project. The representation-theoretic tools needed for these models, including Clebsch-Gordan computations and high-rank tensor decompositions, are by now supported by explicit algorithms and decomposition frameworks [10, 11, 12].

Finally, the UCR archive [13] remains a standard benchmark suite for time-series classification and provides a useful complementary testbed for robustness and sample-efficiency comparisons. The recent survey literature on topological machine learning also offers broader context for where this project sits among topology-aware neural approaches [19, 20, 21, 22, 23].

4 MODELS

We present the models in the same progression as the project itself. The first model introduced was the distance-matrix approach, followed by the PointNet baseline, and finally the TFN family generalized to dimension n .

4.1 DISTANCEMATRIXRAGGEDMODEL

`DistanceMatrixRaggedModel` is one of the new models introduced in this paper for persistence-diagram prediction. Instead of operating on raw coordinates, it uses pairwise distance matrices, which makes it directly tied to geometry and better aligned with the topology prediction task. Because the same row encoder is applied to every row and the outputs are pooled symmetrically, the model behaves like a set function on the rows of the distance matrix. This makes it natural to handle ragged batches while retaining permutation invariance after reindexing of the points.

Algorithm 1: `DistanceMatrixRaggedModel` Forward Pass

Input: Batch of pairwise distance matrices $\mathcal{D} = \{D_1, \dots, D_B\}$.

Output: Predicted topological descriptors.

Process each row of each distance matrix with a shared MLP;

Aggregate row features with summation pooling;

Apply the final prediction head;

return descriptors;

4.2 POINTNET3D

PointNet3D is included as a permutation-invariant baseline. It applies a shared MLP to each point and then aggregates the features by max pooling. This makes it a useful reference model, and it is the canonical PointNet/Deep Sets pattern [1, 2]. The architecture is efficient and robust to permutations, but it does not enforce rotation equivariance.

Algorithm 2: PointNet3D Forward Pass

Input: Batch of ragged point clouds $\mathcal{P} = \{P_1, \dots, P_B\}$.

Output: Predicted descriptors or class scores.

Apply a shared MLP to each point;

Mask padded values;

Max-pool over points in each cloud;

Feed the pooled representation to the final MLP;

return predictions;

4.3 TENSOR FIELD NETWORKS

The **TensorFieldNetwork** class is the main public TFN interface. It implements an $SO(3)$ -equivariant point-cloud classifier for 3D inputs, and it is the closest analogue of the original TFN formulation in the current codebase [8]. Its role in the paper is to serve as the canonical geometric baseline: the model is expressive enough to encode rotations, but still simple enough to interpret as a standard TFN. The higher-dimensional generalization in this project was motivated by a stronger symmetry goal: we wanted robustness to both permutations and rigid isometries, rather than only one of these symmetries at a time. In that sense, the n -dimensional TFN family is the main architectural contribution of the project.

At a schematic level, a TFN layer forms geometric messages from relative positions, couples them with feature tensors, and then projects the result onto irreducible channels using the relevant Clebsch-Gordan coefficients. One can think of the update as

$$m_{ij}^{(\ell)} = \Phi_\ell(d_{ij}) \otimes b_\ell(\hat{r}_{ij}), \quad h_i^{(\ell+1)} = \sum_{j \in \mathcal{N}_k(i)} \Pi_\ell(h_j^{(\ell)} \otimes m_{ij}^{(\ell)}),$$

where Φ_ℓ denotes a radial encoding, b_ℓ denotes a geometric basis, and Π_ℓ denotes the symmetry-adapted projection. This is the algebraic mechanism that makes the network equivariant.

GTTensorFieldNetworkV2 The **GTTensorFieldNetworkV2** model is the most developed TFN variant in the repository. It generalizes the base TFN to arbitrary ambient dimension n and adds the main implementation improvements:

- sparse k-nearest-neighbor neighborhoods instead of dense all-pairs interactions;
- gated nonlinearities, where scalar channels control the activation of higher-order features;
- residual projections per geometric type;
- channel mixing blocks between message-passing layers;
- optional node-attribute injection at the input.

This version is the most practical TFN variant when one wants a stable model that can scale beyond small toy clouds. It is also the version that most directly reflects the project’s move from a 3D-only formulation to a more general n -dimensional symmetry-aware setting.

HierarchicalGTTFN The **HierarchicalGTTFN** model is a multi-scale version of the TFN family. Rather than processing the full point cloud at a single resolution, it builds intermediate geometric summaries and pools them hierarchically. This is useful when the point clouds become larger and a single global pass becomes expensive.

OnEquivariantTensorFieldNetwork The **OnEquivariantTensorFieldNetwork** variant extends the symmetry group from rotations to orthogonal transformations. It is therefore designed to be insensitive not only to rotations, but also to reflections. In practice, this variant is attractive when the task should depend on intrinsic geometry but not on orientation parity.

Common design Despite their differences, these TFN variants share the same core idea:

- build features from distances and directions;
- propagate information through equivariant tensor products;
- preserve geometric structure while learning expressive point-cloud representations.

Algorithm 3: TFN Layer Overview

Input: Point cloud $P \in \mathbb{R}^{N \times n}$ and neighborhood size k .

Output: Updated geometric features.

Build a k -NN graph on the point cloud;

Encode pairwise distances with an RBF expansion;

Compute directional geometric features from relative vectors;

Propagate messages through equivariant tensor products;

Apply gating, residual connections, and channel mixing;

return updated features;

At a high level, the TFN variants differ mainly in how much structure they add around this core layer: the base **TensorFieldNetwork** keeps the architecture close to the original TFN idea, while **GTTensorFieldNetworkV2** adds the most aggressive engineering improvements, and the hierarchical / orthogonal variants adapt the same backbone to larger or more symmetry-sensitive settings.

5 EXPERIMENTAL SETUP

The experiments are organized around three complementary settings: synthetic point-cloud tasks, UCR time-series benchmarks recast as point-cloud problems, and notebook-driven exploratory analyses. The common objective is to compare geometric inductive biases under clean and perturbed inputs.

5.1 EVALUATION PROTOCOL

We evaluate the models using three metrics:

- classification accuracy on clean and noisy data;
- isometry robustness, measured by the variation of the output under rigid transformations;
- permutation robustness, measured by the variation of the output under point reordering.

The TFN models are expected to perform best on isometry-related tests because they build geometric symmetry into the architecture. PointNet-style models are expected to be robust to permutations. DistanceMatrixRaggedModel benefits from the geometric prior induced by pairwise distances, which is particularly relevant for persistence-oriented tasks.

5.2 SYNTHETIC POINT-CLOUD BENCHMARKS

The synthetic experiments are the most direct way to study topological prediction. They use generated point clouds in which the target depends on the underlying shape, such as the number of circles or the presence of topological loops. These tasks are useful because they isolate the geometric part of the learning problem: the model must infer topology from noisy point samples rather than from hand-crafted features.

In the earlier stages of the project, synthetic experiments were used to compare RipsNet-style persistence prediction with PointNet-like and distance-based models. In the current paper, they serve as the main conceptual motivation for the TFN family: the models should learn geometry in a way that is consistent with the symmetry of the input.

5.3 UCR BENCHMARKS

The repository also contains a large experiment suite for UCR time-series datasets. In this setting, each time series is treated as a geometric object and benchmarked across a set of 21 datasets, including CBF, ECG200, ECG5000, GunPoint, ItalyPowerDemand, Plane, PowerCons, SonyAIBORobotSurface1, SonyAIBORobotSurface2, and TwoLeadECG. This benchmark family comes from the UCR archive, which is a standard reference for reproducible time-series evaluation [13].

The UCR pipeline is useful for two reasons. First, it provides a standardized benchmark suite with widely varying sample sizes, sequence lengths, and class counts. Second, it allows us to compare TFN variants, PointNet baselines, and the distance-matrix model under a common training and evaluation script.

The dataset metadata recorded in the repository shows that these tasks vary considerably in scale. Some sets are tiny, such as SonyAIBORobotSurface1, while others are much larger, such as ECG5000 and ChlorineConcentration. This diversity is important because it reveals whether a model is robust only on small toy data or whether it generalizes across benchmark regimes.

5.4 TRAINING SCRIPTS

The core training entry point is `expos/train_nn.py`. This script loads precomputed persistence-vector datasets, prepares model-specific geometric inputs, and trains a selected architecture with mini-batch

optimization. It also caches TFN geometry on disk so that k-NN neighborhoods and basis tensors do not need to be recomputed for every run.

Several design details matter in practice:

- TFN models use precomputed geometry and can optionally use Gaussian smoothing;
- `DistanceMatrixRaggedModel` is fed pairwise distances rather than raw coordinates;
- `PointNet3D` pads point clouds and applies max pooling;
- dataset-specific hyperparameters are overridden for certain UCR tasks to keep the models stable.

The ablation script `expes/train_ablation.py` is used to measure how performance changes when the fraction of training data is reduced. This is useful for comparing model efficiency and sample complexity across TFN variants, baselines, and distance-based models.

5.5 EXPERIMENT DIRECTORY

The `expes/` directory is the main experiment hub. It contains the training scripts, ablation scripts, result-processing scripts, and the job-launching shell scripts used for large sweeps. The structure is intentionally modular:

- `expes/train_nn.py` handles the standard training workflow;
- `expes/train_ablation.py` handles the data-fraction ablations;
- `expes/results/analyze_models.py` summarizes performance across the UCR suite;
- the shell scripts in `expes/env/` launch dataset- and model-specific jobs on different machines.

5.6 NOTEBOOK WORKFLOW

The notebook workflow complements the scripted experiments. In practice, notebooks were used for exploratory analysis, visual inspection of point clouds and embeddings, and rapid sanity checks while the models were being developed. They were especially useful for checking the ragged batching logic, the geometry preprocessing, and the qualitative behavior of the learned embeddings before freezing the settings into the scripted runs.

Taken together, the scripts and notebooks form a two-level experimental workflow: the notebooks support fast iteration and interpretation, while the `expes/` scripts provide reproducible benchmark runs.

6 RESULTS

The main qualitative conclusion is that the TFN family now represents the central contribution of the paper. The newer models are organized around geometric symmetry:

- TFN variants for rotation and reflection-aware learning;
- PointNet3D for permutation-invariant baseline comparisons;
- DistanceMatrixRaggedModel for persistence-oriented prediction from pairwise geometry.

In practice, the TFN models are the most principled choice when the input geometry itself matters. The PointNet baseline remains valuable as a simple reference, and the distance-matrix model provides a second geometric baseline closer to the persistence-diagram task.

7 CONCLUSION

This paper presents a TFN-centered study of geometric deep learning for point clouds. The most important contribution is the addition of several TFN-based models, together with a persistence-oriented `DistanceMatrixRaggedModel`.

The paper now reflects the current direction of the codebase:

- TFN models are the main focus;
- `DistanceMatrixRaggedModel` is treated as a genuine new model, not just an auxiliary baseline;
- the older report-style narrative has been removed.

REFERENCES

- [1] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Póczos, R. Salakhutdinov, and A. J. Smola. Deep Sets. *NeurIPS*, 2017.
- [2] C. R. Qi, H. Su, K. Mo, and L. J. Guibas. PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation. *CVPR*, 2017.
- [3] C. R. Qi, L. Yi, H. Su, and L. J. Guibas. PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space. *NeurIPS*, 2017.
- [4] Y. Wang, Y. Sun, Z. Liu, S. E. Sarma, M. M. Bronstein, and J. M. Solomon. Dynamic Graph CNN for Learning on Point Clouds. *ACM Transactions on Graphics*, 38(5), 2019.
- [5] H. Adams et al. Persistence Images: A Stable Vector Representation of Persistent Homology. *Journal of Machine Learning Research*, 18(1), 2017.
- [6] P. Bubenik. Statistical Topological Data Analysis Using Persistence Landscapes. *Journal of Machine Learning Research*, 16, 2015.
- [7] D. Cohen-Steiner, H. Edelsbrunner, and J. Harer. Stability of Persistence Diagrams. *Discrete & Computational Geometry*, 37:103–120, 2007.
- [8] N. Thomas et al. Tensor Field Networks: Rotation- and Translation-Equivariant Neural Networks for 3D Point Clouds. arXiv preprint arXiv:1802.08219, 2018.
- [9] E. Levin and M. Díaz. Any-dimensional Equivariant Neural Networks. *AISTATS*, 2024.
- [10] A. Ibort, A. López-Yela, and J. Moro. A New Algorithm for Computing Branching Rules and Clebsch-Gordan Coefficients of Unitary Representations of Compact Groups. *Journal of Mathematical Physics*, 58(10), 2017.
- [11] A. Alex, M. Kalus, A. Huckleberry, and J. von Delft. A Numerical Algorithm for the Explicit Calculation of $SU(N)$ and $SL(N, \mathbb{C})$ Clebsch-Gordan Coefficients. *Journal of Mathematical Physics*, 52(2), 2011.
- [12] S. Shao, Y. Li, Z. Lin, and Q. Cui. High-Rank Irreducible Cartesian Tensor Decomposition and Bases of Equivariant Spaces. arXiv preprint arXiv:2412.18263, 2024.
- [13] H. A. Dau et al. The UCR Time Series Archive. *Data Mining and Knowledge Discovery*, 33:599–661, 2019.
- [14] T. de Surrél et al. RipsNet: a general architecture for fast and robust estimation of the persistent homology of point clouds. *Proceedings of Topological, Algebraic, and Geometric Learning Workshops*, 2022.
- [15] M. Carrière et al. PersLay: A Neural Network Layer for Persistence Diagrams and New Graph Topological Signatures. *AISTATS*, 2020.
- [16] K. Kim et al. PLLay: Efficient Topological Layer based on Persistence Landscapes. arXiv preprint, 2020.

- [17] A. Som et al. Perturbation Robust Representations of Topological Persistence Diagrams. arXiv preprint, 2018.
- [18] M. Carrière, M. Cuturi, and S. Oudot. Optimizing Persistent Homology-Based Functions. *ICML*, 2017.
- [19] F. Hensel, M. Moor, and B. Rieck. A Survey of Topological Machine Learning Methods. *Frontiers in Artificial Intelligence*, 2021.
- [20] M. Moor et al. Topological Autoencoders. *ICML*, 2020.
- [21] N. Madhu et al. Topology Preserving Self-Supervised Simplicial Representation Learning. *NeurIPS*, 2023.
- [22] J. Shin et al. Line Graph Vietoris-Rips Persistence Diagram for Topological Graph Representation Learning. *JMLR*, 2024.
- [23] Learning Persistent Homology of 3D Point Clouds. *Computers & Graphics*, 2022.